

KARAOKE VIDEO GENERATOR

Final Sprint Review Presentation

Team: Aruhant Mehta, Mark Rozin, Thomas Zhang, William Cagas

Technical Lead: William Cagas

Project Sponsor: Paul Ward

Date: December 2025

Course: SE-101

Slide 1: Title Slide



Karaoke Video Generator

Final Sprint Review

Automated karaoke video creation from YouTube links

Team Members:

- Aruhant Mehta (Lyrics Intelligence)
- Mark Rozin (Audio Processing)
- Thomas Zhang (Audio Acquisition)
- William Cagas (Video Rendering & Technical Lead)

Project Sponsor: Paul Ward

Timeline: November 4 - December 2, 2025 (4 weeks)

Slide 2: Project Overview

Problem Statement

Creating karaoke videos is surprisingly manual:

- Extract audio from YouTube
- Remove vocals using complex tools
- Find and sync lyrics manually
- Render video with text overlays

Our Solution

Fully automated pipeline that converts any YouTube song into a karaoke video with synchronized lyrics in under 5 minutes.

Key Stakeholders

- **Project Sponsor:** Paul Ward
 - **End Users:** Content creators, karaoke enthusiasts, musicians
 - **Advisors:** Course Instructor, TA Mentors
-

Slide 3: Sprint Summary

Sprint 1: Foundation & Research (Nov 4-11)

- Project architecture & setup
- Research on audio processing, lyrics APIs, video rendering
- Module structures created
- Documentation framework established

Sprint 2: Core Implementation (Nov 11-18)

- YouTube audio downloader implemented
- Demucs vocal separation pipeline
- Lyrics fetching & LRC file generation
- FastAPI backend setup

Sprint 3: Integration & Synchronization (Nov 18-25)

- Lyrics timestamp synchronization
- Audio processing integration
- Video rendering foundation
- Frontend UI implementation

Sprint 4: Integration, Testing & Demo (Nov 25-Dec 2)

- End-to-end integration
 - Bug fixes & performance optimization
 - Comprehensive testing
 - Demo preparation
-

Slide 4: Increment Summary

What Was Delivered

Sprint 1 Deliverables:

- ☒ Project architecture document
- ☒ Module structures (audio acquisition, processing, lyrics, video)
- ☒ Research completed on all core technologies
- ☒ Development environment setup

Sprint 2 Deliverables:

- ☒ YouTube → MP3 conversion with error handling
- ☒ Demucs vocal separation pipeline
- ☒ Spleeter fallback implementation
- ☒ Audio quality optimization (normalization, noise reduction)
- ☒ Lyrics fetching from APIs
- ☒ LRC file generation

Sprint 3 Deliverables:

- ☒ Lyrics timestamp synchronization (90% accuracy)
- ☒ Audio processing integration with acquisition
- ☒ Video rendering engine (720p)
- ☒ Synchronized lyrics overlay
- ☒ React frontend with TypeScript

Sprint 4 Deliverables:

- ☒ Complete end-to-end pipeline
- ☒ Performance optimization (40% faster rendering)
- ☒ Bug fixes & error handling
- ☒ Demo-ready application
- ☒ Comprehensive documentation

Slide 5: Velocity Metrics

Sprint Velocity Overview

Sprint	Planned	Completed	Velocity	Completion Rate
Sprint 1	21	18	18	86%
Sprint 2	26	22	22	85%
Sprint 3	28	25	25	89%
Sprint 4	24	23	23	96%
Total	99	88	88	89%

Key Observations

- Steady velocity growth: 18 → 22 → 25 → 23
- Improving completion rate: 86% → 96%
- Consistent delivery: 88 story points completed over 4 sprints
- High final sprint completion: 96% in Sprint 4

Slide 6: Success Criteria Achievement

Project Success Criteria

Criterion	Target	Achieved	Status
Lyric Accuracy	≥ 90%	~90%	✓ MET
Video Render Time	< 5 min (3-min song)	< 5 min	✓ MET
Functional MVP	Delivered by Week 4	Delivered	✓ MET

Additional Achievements

- ✓ Successfully generates karaoke videos for ~90% of tested songs
- ✓ Performance optimization: 40% faster rendering than Sprint 3
- ✓ Robust error handling and fallback mechanisms
- ✓ Clean, modern UI with real-time progress indicators

Slide 7: Technical Highlights

Technology Stack

Backend:

- Python 3.x
- FastAPI (REST API)
- Demucs (AI vocal separation)
- Spleeter (fallback separation)
- FFmpeg (video rendering)
- yt-dlp (YouTube audio download)

Frontend:

- React + TypeScript
- Vite (build tool)
- shadcn-ui (UI components)

APIs & Services:

- LRCLIB API (synchronized lyrics)
- Genius API (lyrics fetching)
- WhisperX (timestamp generation fallback)

Architecture Highlights

- **Modular design:** Clear separation of concerns
 - **Async processing:** Non-blocking operations for long-running tasks
 - **Error resilience:** Multiple fallback mechanisms
 - **Performance optimized:** Hardware acceleration where available
-

Slide 8: Challenges & Solutions

Major Challenges Faced

Challenge 1: Lyrics Synchronization Accuracy

- **Problem:** Timestamp accuracy varied wildly initially
- **Solution:** Implemented hybrid approach using pre-synced LRC files with WhisperX fallback

- **Result:** Achieved ~90% accuracy for most pop songs

Challenge 2: Video Rendering Performance

- **Problem:** Initial rendering was ~2x song duration
- **Solution:** Optimized FFmpeg calls, hardware acceleration, removed MoviePy overhead
- **Result:** 40% performance improvement, meeting <5 min target

Challenge 3: Integration Complexity

- **Problem:** Interface mismatches between modules
- **Solution:** Early API contract definition, refactoring during Sprint 3
- **Result:** Smooth end-to-end integration

Challenge 4: YouTube Rate Limiting

- **Problem:** Audio download failures due to rate limits
 - **Solution:** Implemented yt-dlp with fallback mechanisms
 - **Result:** Robust audio acquisition
-

Slide 9: Stakeholder Feedback

Feedback from Demo Session

From Professors:

- "Impressed with the real-time lyrics highlighting"
- "Smooth demo execution"
- "Well-integrated system"

From TAs:

- "Positive feedback on technical implementation"
- "Good use of open-source tools"
- "Clean code structure"

Project Success Metrics

-  **90% success rate** for karaoke video generation
-  **Performance targets met** (render time < 5 min)
-  **Accuracy targets met** (lyric sync ≥ 90%)
-  **All core features delivered** on schedule

User Experience

- Clean, intuitive interface
 - Real-time progress tracking
 - Helpful error messages
 - Professional video output quality
-

Slide 10: Lessons Learned

Technical Learnings

- **Audio/video processing is computationally expensive** – always consider performance early
- **API rate limits are a real constraint** – caching strategies are essential
- **Machine learning models require careful tuning** – WhisperX needed optimization
- **Modular architecture pays off** – made integration smoother

Process Learnings

- **Daily standups prevent misunderstandings** – regular communication is critical
- **Code reviews catch bugs and spread knowledge** – worth the time investment
- **Playing to individual strengths leads to better outcomes** – team collaboration
- **Iterative development and adaptation** – key to success under tight deadlines

What We'd Do Differently

- Start with simpler MVP and iterate (tried to do too much initially)
 - Allocate more time for integration testing between sprints
 - Set up CI/CD pipeline from day one
 - Have clearer API contracts before implementation
 - Schedule regular pair programming sessions
-

Slide 11: Future Enhancements

Potential Improvements

Short-term:

- Manual timestamp adjustment for edge cases

- Batch processing UI improvements
- Additional video background options
- Enhanced error recovery

Medium-term:

- Multi-language support
- Advanced visual effects (color transitions, animations)
- User authentication and saved projects
- Cloud hosting for scalability

Long-term:

- Mobile app version
- Real-time collaboration features
- Commercial licensing support
- AI-powered lyric accuracy improvements

Open Source Potential

- Consider open-sourcing the project after course completion
 - Could benefit the karaoke and content creation community
-

Slide 12: Demo Highlights

Live Demo Script (2 minutes)

[0:00-0:15] Hook

"Creating karaoke videos is surprisingly manual — you need to extract audio, remove vocals, find lyrics, sync them, and render a video. Our project automates that entire pipeline."

[0:15-0:35] What it does

"This is the Karaoke Video Generator. Given any YouTube link, our system automatically converts the song into a karaoke video with synchronized lyrics, ready to use or upload."

[0:35-1:05] How it works

"Under the hood, the system is a fully automated pipeline:

- Download audio from YouTube and convert to MP3
- Use AI-based vocal separation to isolate instrumental track
- Extract lyrics, generate timestamps, produce LRC file
- Render 720p karaoke video with real-time lyric overlays"

[1:05-1:30] Team & architecture

"Our team split responsibilities across the pipeline:

- Thomas: Audio acquisition
- Mark: Vocal separation
- Aruhant: Lyric intelligence and synchronization
- William: Video rendering and FFmpeg integration

Everything is written in Python using open-source tools."

[1:30-1:50] Demo moment

"Here's an example output. You can see the vocals removed, lyrics perfectly synced, and the video rendered automatically in under five minutes for a three-minute song."

[1:50-2:00] Wrap-up

"In four weeks, we delivered a fully functional MVP that automates karaoke video creation end-to-end, meeting our performance and accuracy goals. Thank you."

Slide 13: Q&A

Questions?

Contact Information:

- Technical Lead: William Cagas
- Repository: [GitHub/GitLab link]
- Documentation: [/docs](#) folder

Thank you for your attention!

Appendix: Team Contributions

Individual Contributions

Thomas Zhang (Audio Acquisition)

- YouTube downloader implementation
- MP3 conversion with quality options
- Error handling and fallback mechanisms

Mark Rozin (Audio Processing)

- Demucs vocal separation pipeline
- Spleeter fallback implementation
- Audio quality optimization (normalization, noise reduction)
- Audio processing integration

Aruhant Mehta (Lyrics Intelligence)

- Lyrics fetching from APIs
- Title normalization system
- Lyrics timestamp synchronization
- LRC file generation

William Cagas (Video Rendering & Technical Lead)

- FFmpeg video rendering engine
- Synchronized lyrics overlay
- Frontend UI implementation
- End-to-end integration
- Project management and documentation

End of Presentation